SystemVerilog中父类与子类

1.父类和子类中同时定义同样的变量

- **When you redefine a variable in a subclass that already exists in the parent class, SystemVerilog treats them as separate variables.** This means that both the parent and subclass will maintain their own versions of the variable with their respective constraints.
- Example Illustration(父类和子类同时定义同样的变量和不同名 constraint)
- Parent Class

```
// lvc_apb_master_base_sequence.sv
class lvc_apb_master_base_sequence extends uvm_sequence
#(lvc_apb_transfer);
rand bit [31:0] addr;

constraint addr_zero {
addr == 32'd0;
}

// Other members...
endclass
```

- Subclass

```
// lvc_apb_master_burst_write_sequence.sv
class lvc_apb_master_burst_write_sequence extends
lvc_apb_master_base_sequence;
rand bit [31:0] addr; // This hides the parent's 'addr'
rand bit [31:0] data[];
lvc_apb_trans_status trans_status;

constraint cstr {
soft data.size() inside {4, 8, 16, 32};
foreach(data[i]) soft data[i] == addr + (i << 2);
}

// Other members...
endclass
```

- Randomization Behavior
  - When you create an instance of lvc_apb_master_burst_write_sequence and call randomize(), the following happens:
  - Parent's addr: Set to 0 due to addr_zero constraint.

    - Subclass's addr: Randomized based on the subclass's cstr constraints.
- Accessing Variables
    - Parent's addr: Accessed using super.addr within the subclass.
    - Subclass's addr: Accessed directly as addr within the subclass.
- Recommendations
    - Avoid Hiding Variables:
      To prevent confusion, avoid redefining variables in subclasses. Instead, consider extending the constraints if needed.
    - Use Different Names:
      If you must have different behaviors, use distinct variable names in the parent and subclass.
    - Constraint Management:
      Leverage extends and override keywords to manage constraints without hiding variables.
- Conclusion
  The constraints defined in the parent class will still apply to the parent class's version of addr. However, the subclass has its own separate addr with its own constraints.
  After randomization:
    - Parent's addr: Will be 0.
    - Subclass's addr: Will be randomized based on the subclass's constraints.
      This separation allows both classes to enforce their own constraints independently.
- SystemVerilog manages variables and constraints in class hierarchies

Variable Scope and Hiding

Understanding Variable Scope

In SystemVerilog, classes support inheritance, allowing subclasses to inherit variables and methods from their parent classes. Variables can be public, protected, or private, determining their accessibility in subclasses.

Variable Hiding in Subclasses

When a subclass defines a variable with the same name as a variable in its parent class, the subclass's variable hides the parent's variable. This means:

- Parent Variable: Accessible within the parent class and via `super` in the subclass.
- Subclass Variable: Accessible within the subclass without qualifiers.
- Key Point: Both variables exist independently; they are not the same variable.

Example Illustration

- Parent's `addr`: Exists separately and is constrained to 0.
- Subclass's `addr`: A new variable with its own constraints.

Constraint Inheritance and Overriding

Constraint Application

When randomization occurs on an object, all applicable constraints from the class hierarchy are considered: Parent Class Constraints and Subclass Constraints.

Overriding Constraints

Constraints in the subclass do not override parent constraints unless explicitly managed. Instead, if variables are hidden, constraints apply to their respective variables independently.

Example Behavior

Using the previous classes:

- Parent's addr Constraint: `addr == 0`
- Subclass's addr Constraint: Defined in `cstr` (e.g., depends on data)

When an instance of `lvc_apb_master_burst_write_sequence` is randomized:

- Parent's addr: Set to 0
- Subclass's addr: Randomized based on subclass constraints

This occurs because both `addr` variables exist separately due to variable hiding.

Instance Differentiation

Accessing Parent and Subclass Variables

Within the subclass:

- Subclass's addr: Accessed directly as `addr`
- Parent's addr: Accessed using `super.addr`

Practical Example

```
virtual task example_task();
$display("Subclass addr: %0d", addr);
// Outputs subclass's addr
$display("Parent addr: %0d", super.addr);
// Outputs parent's addr (0)
endtask
```

Randomization Example

```
lvc_apb_master_burst_write_sequence seq = new();

// Randomize the subclass's variables
if (!seq.randomize()) begin
`uvm_error("Randomization Failed", "Failed to randomize the sequence")
end else begin
// Display values
```

```
$display("Subclass addr after randomization: %0d", seq.addr);
// Randomized value
$display("Parent addr after randomization: %0d", seq.super.addr);
// cannot access parent attribute if it is not inside the class's function
end
```

Output:

```
Subclass addr after randomization: 100
Parent addr after randomization: 0
```

- 父类和子类同时定义同样的变量和不同名 constraint / 同名 constraint
- 父类和子类的 constraint 分别作用于各自的变量，不会互相影响，如果通过子类对象 randomize，父类的变量不会被 ranomize

```
module junk();
class base;
rand bit [15:0] a,b;

function new();
endfunction // new

constraint ct_a {a inside {[0:9]};}
endclass // base

class extended extends base;
rand bit [15:0] a,b;
function new();
   super.new(); // Call parent constructor
endfunction // new

constraint ct_a  {a inside {[10:19]};}

function int get_parent_value();
 return super.a;
endfunction
endclass // extended

initial begin
extended extended_obj;
base base_obj;
extended_obj = new();
base_obj = new();

if(!extended_obj.randomize()) $display("Randomization Failed for
extended_obj");
if(!base_obj.randomize()) $display("Randomization Failed for base_obj");

$display("Extended class a: %0d", extended_obj.a);
$display("Base class a: %0d", base_obj.a);
$display("Parent's a accessed from extended: %0d",
extended_obj.get_parent_value());
```

```
end
endmodule // junk

//Extended class a: 12
//Base class a: 0
//Parent's a accessed from extended: 55225
```

- 父类和子类同时定义同样的变量和不同名 constraint / 同名 constraint，创建父类的时候使用多态
- 对父类对象 randomize，实质上还是对子类 randomize，父类的变量不会被 randomize，通过父类句柄只能访问父类变量

```
module junk();
class base;
rand bit [15:0] a,b;

function new();
endfunction // new

constraint ct_a {a inside {[0:9]};}
endclass // base

class extended extends base;
rand bit [15:0] a,b;
function new();
   super.new(); // Call parent constructor
endfunction // new

constraint ct_a  {a inside {[10:19]};}

function int get_parent_value();
 return super.a;
endfunction
endclass // extended

initial begin
extended extended_obj;
base base_obj;
extended_obj = new();
base_obj = extended_obj;

//if(!extended_obj.randomize()) $display("Randomization Failed for
extended_obj");
if(!base_obj.randomize()) $display("Randomization Failed for base_obj");

$display("Extended class a: %0d", extended_obj.a);
$display("Base class a: %0d", base_obj.a);
$display("Parent's a accessed from extended: %0d",
extended_obj.get_parent_value());
end
endmodule // junk
```

```
//Extended class a: 12
//Base class a: 55225
//Parent's a accessed from extended: 55225
```

```
module junk();
class base;
rand bit [15:0] a,b;

function new();
endfunction // new

constraint ct_a {a inside {[0:9]};}
endclass // base

class extended extends base;
rand bit [15:0] a,b;
function new();
   super.new(); // Call parent constructor
endfunction // new

constraint ct_a  {a inside {[10:19]};}

function int get_parent_value();
 return super.a;
endfunction
endclass // extended

initial begin
extended extended_obj;
base base_obj;
extended_obj = new();
base_obj = extended_obj;

if(!extended_obj.randomize()) $display("Randomization Failed for
extended_obj");
//if(!base_obj.randomize()) $display("Randomization Failed for base_obj");

$display("Extended class a: %0d", extended_obj.a);
$display("Base class a: %0d", base_obj.a);
$display("Parent's a accessed from extended: %0d",
extended_obj.get_parent_value());
end
endmodule // junk

//Extended class a: 12
//Base class a: 55225
//Parent's a accessed from extended: 55225
```

- 只有父类定义变量且不同名 constraint
- 父类对象 randomize: 父类正常使用父类 constraint，子类不会被 randomize

```
module junk();
class base;
rand bit [15:0] a,b;

function new();
endfunction // new

constraint ct_a {a inside {[2:9]};}
endclass // base

class extended extends base;
//rand bit [15:0] a,b;
function new();
   super.new(); // Call parent constructor
endfunction // new

constraint ct_b  {a inside {[10:19]};}

function int get_parent_value();
 return super.a;
endfunction
endclass // extended

initial begin
extended extended_obj;
base base_obj;
extended_obj = new();
base_obj = new();

//if(!extended_obj.randomize()) $display("Randomization Failed for
extended_obj");
if(!base_obj.randomize()) $display("Randomization Failed for base_obj");

$display("Extended class a: %0d", extended_obj.a);
$display("Base class a: %0d", base_obj.a);
$display("Parent's a accessed from extended: %0d",
extended_obj.get_parent_value());
end
endmodule // junk

//Extended class a: 0
//Base class a: 2
//Parent's a accessed from extended: 0
```

- 子类对象 randomize: 两个 constraint 同时作用于同一个对象，会产生 conflict

```
module junk();
class base;
rand bit [15:0] a,b;

function new();
endfunction // new
```

```
constraint ct_a {a inside {[0:9]};}
endclass // base

class extended extends base;
//rand bit [15:0] a,b;
function new();
   super.new(); // Call parent constructor
endfunction // new

constraint ct_b  {a inside {[10:19]};}

function int get_parent_value();
 return super.a;
endfunction
endclass // extended

initial begin
extended extended_obj;
base base_obj;
extended_obj = new();
base_obj = new();

if(!extended_obj.randomize()) $display("Randomization Failed for
extended_obj");
if(!base_obj.randomize()) $display("Randomization Failed for base_obj");

$display("Extended class a: %0d", extended_obj.a);
$display("Base class a: %0d", base_obj.a);
$display("Parent's a accessed from extended: %0d",
extended_obj.get_parent_value());
end
endmodule // junk

/*
constraint ct_a    // (from this) (constraint_mode = ON) (testbench.sv:9)
{
(a inside {[0:9]});
}
constraint ct_b    // (from this) (constraint_mode = ON) (testbench.sv:18)
{
(a inside {[10:19]});
}
*/
```

- 只有父类定义变量且同名 constraint
- 父类对象 randomize: 父类正常使用父类 constraint，子类不会被 randomize
- 子类对象 randomize: 子类的 constraint override 父类 constraint

```
module junk();
class base;
rand bit [15:0] a,b;

function new();
```

```
endfunction // new

constraint ct_a {a inside {[2:9]};}
endclass // base

class extended extends base;
//rand bit [15:0] a,b;
function new();
   super.new(); // Call parent constructor
endfunction // new

constraint ct_a  {a inside {[10:19]};}

function int get_parent_value();
 return super.a;
endfunction
endclass // extended

initial begin
extended extended_obj;
base base_obj;
extended_obj = new();
base_obj = new();

if(!extended_obj.randomize()) $display("Randomization Failed for
extended_obj");
if(!base_obj.randomize()) $display("Randomization Failed for base_obj");

$display("Extended class a: %0d", extended_obj.a);
$display("Base class a: %0d", base_obj.a);
$display("Parent's a accessed from extended: %0d",
extended_obj.get_parent_value());
end
endmodule // junk

//Extended class a: 12
//Base class a: 2
//Parent's a accessed from extended: 12
```

2.父类和子类中同时定义同样的方法

- **When a subclass defines a method (task or function) with the same name as one in the parent class, the subclass method overrides the parent method.** Unlike variables, methods are not "hidden" — the subclass method replaces the parent's method polymorphically when called through a subclass handle. To call the parent's version, use `super.method_name()`.
- 父类和子类同时定义同名方法（非 virtual）
- 非 virtual 方法：方法的调用取决于句柄的静态类型，而不是对象的实际类型。通过父类句柄调用时执行父类方法，通过子类句柄调用时执行子类方法。

```
module junk();
class base;
function void print_info();
   $display("Base class print_info called");
endfunction
endclass

class extended extends base;
function void print_info();
   $display("Extended class print_info called");
endfunction
endclass

initial begin
extended ext_obj = new();
base base_obj = new();
base base_handle;

base_handle = ext_obj; // 多态赋值

ext_obj.print_info();    // 调用子类方法
base_obj.print_info();   // 调用父类方法
base_handle.print_info(); // 静态绑定：调用父类方法！
end
endmodule

//Extended class print_info called
//Base class print_info called
//Base class print_info called   <-- 注意：非virtual，静态绑定，调用父类
```

- 父类和子类同时定义同名方法（virtual）
- **virtual 方法**：方法的调用取决于对象的实际类型（动态绑定）。即使通过父类句柄调用，也会执行子类方法。这是实现多态的核心机制。

```
module junk();
class base;
virtual function void print_info();
   $display("Base class print_info called");
endfunction
endclass

class extended extends base;
virtual function void print_info(); // virtual 可省略，继承自父类
   $display("Extended class print_info called");
endfunction
endclass

initial begin
extended ext_obj = new();
base base_obj = new();
base base_handle;
```

```
base_handle = ext_obj; // 多态赋值

ext_obj.print_info();     // 调用子类方法
base_obj.print_info();    // 调用父类方法
base_handle.print_info(); // 动态绑定：调用子类方法！
end
endmodule

//Extended class print_info called
//Base class print_info called
//Extended class print_info called  <-- 注意：virtual，动态绑定，调用子类
```

- 在子类方法中使用 `super` 调用父类方法
- 子类覆写方法时，可以通过 `super.method_name()` 显式调用父类的同名方法，常用于在扩展父类行为的同时保留父类逻辑。

```
module junk();
class base;
virtual function void print_info();
   $display("Base class print_info called");
endfunction
endclass

class extended extends base;
virtual function void print_info();
   super.print_info(); // 先调用父类方法
   $display("Extended class print_info called");
endfunction
endclass

initial begin
extended ext_obj = new();
ext_obj.print_info();
end
endmodule

//Base class print_info called
//Extended class print_info called
```

- 只有父类定义方法，子类未覆写
- 子类继承父类的方法，子类对象可以直接调用。子类中可以用 `super.method_name()` 调用，效果相同。

```
module junk();
class base;
virtual function void print_info();
   $display("Base class print_info called");
endfunction
endclass
```

```
class extended extends base;
// 未定义 print_info，继承自父类
function new();
   super.new();
endfunction
endclass

initial begin
extended ext_obj = new();
base base_obj = new();

ext_obj.print_info();  // 调用继承的父类方法
base_obj.print_info(); // 调用父类方法
end
endmodule

//Base class print_info called
//Base class print_info called
```

- Summary

场景	非 virtual 方法	virtual 方法
子类句柄调用	子类方法	子类方法
父类句柄调用（指向子类对象）	父类方法（静态绑定）	子类方法（动态绑定）
`super.method()`	调用父类方法	调用父类方法

3.父类和子类变量的互相访问

- **Variable accessibility between parent and subclass is governed by access modifiers: `local`, `protected`, and public (default).** Understanding these modifiers is essential to correctly reading and writing variables across the class hierarchy.
- 访问修饰符总结

修饰符	父类内部	子类内部	类外部（句柄访问）
无（public）	✓	✓	✓
`protected`	✓	✓	✗
`local`	✓	✗	✗

- public 变量（默认）：父类、子类、外部均可访问

```
module junk();
class base;
bit [15:0] pub_var; // public，默认
```

```
function new();
   pub_var = 16'hAAAA;
endfunction
endclass

class extended extends base;
function void show();
   $display("Accessing parent public var in subclass: %0h", pub_var); //
直接访问
endfunction
endclass

initial begin
extended ext_obj = new();
ext_obj.show();
$display("Accessing public var from outside: %0h", ext_obj.pub_var); // 外
部访问
end
endmodule

//Accessing parent public var in subclass: aaaa
//Accessing public var from outside: aaaa
```

- `protected` 变量：父类和子类内部可访问，类外部不可访问

```
module junk();
class base;
protected bit [15:0] prot_var;

function new();
   prot_var = 16'hBBBB;
endfunction
endclass

class extended extends base;
function void show();
   $display("Accessing parent protected var in subclass: %0h",
prot_var); // 合法
endfunction

function void modify();
   prot_var = 16'hCCCC; // 子类可以修改
endfunction
endclass

initial begin
extended ext_obj = new();
ext_obj.show();
ext_obj.modify();
ext_obj.show();
// $display("%0h", ext_obj.prot_var); // 非法！类外部不可访问
end
endmodule
```

```
//Accessing parent protected var in subclass: bbbb
//Accessing parent protected var in subclass: cccc
```

- `local` 变量：仅父类内部可访问，子类和外部均不可访问

```
module junk();
class base;
local bit [15:0] local_var;

function new();
   local_var = 16'hDDDD;
endfunction

function bit [15:0] get_local();
   return local_var; // 父类内部访问合法
endfunction
endclass

class extended extends base;
function void show();
   // $display("%0h", local_var); // 非法！子类不可访问 local 变量
   $display("Accessing parent local var via getter: %0h", get_local()); //
通过方法访问
endfunction
endclass

initial begin
extended ext_obj = new();
ext_obj.show();
$display("Via getter from outside: %0h", ext_obj.get_local()); // 通过公共方
法访问
end
endmodule

//Accessing parent local var via getter: dddd
//Via getter from outside: dddd
```

- 子类访问父类变量：使用 `super`
- 在子类中，当子类定义了与父类同名的变量（variable hiding），需要用 `super.var_name` 访问父类变量。如果没有同名变量，直接访问即可。

```
module junk();
class base;
bit [15:0] a;
function new();
   a = 16'h1111;
endfunction
endclass

class extended extends base;
```

```
bit [15:0] a; // 隐藏父类的 a

function new();
   super.new();
   a = 16'h2222;
endfunction

function void show();
   $display("Subclass a:      %0h", a);       // 子类的 a
   $display("Parent a (super): %0h", super.a); // 父类的 a
endfunction
endclass

initial begin
extended ext_obj = new();
ext_obj.show();
end
endmodule

//Subclass a:       2222
//Parent a (super): 1111
```

- 父类句柄访问子类变量
- 父类句柄**不能**直接访问子类中新定义的变量，即使该句柄实际指向子类对象。若需要访问，必须先将句柄向下转型（`$cast`）为子类类型。

```
module junk();
class base;
bit [15:0] a;
function new();
   a = 16'h1111;
endfunction
endclass

class extended extends base;
bit [15:0] b; // 子类新增变量

function new();
   super.new();
   b = 16'h2222;
endfunction
endclass

initial begin
extended ext_obj = new();
base base_handle;
extended cast_obj;

base_handle = ext_obj;

$display("base_handle.a = %0h", base_handle.a);   // 合法，访问父类变量
// $display("base_handle.b = %0h", base_handle.b); // 非法！父类句柄不能访问子
```

```
类变量

// 通过 $cast 向下转型后可访问
if ($cast(cast_obj, base_handle)) begin
   $display("After $cast, cast_obj.b = %0h", cast_obj.b); // 合法
end
end
endmodule

//base_handle.a = 1111
//After $cast, cast_obj.b = 2222
```

4.父类和子类方法的互相访问

- **Method accessibility follows the same `local` / `protected` / public rules as variables.** In addition, the `super` keyword allows a subclass to explicitly call the parent's version of a method, and `$cast` allows a parent handle to be downcast to access subclass-only methods.
- 子类调用父类方法：直接继承调用 / `super` 显式调用
- 子类默认继承所有非 `local` 的父类方法，可直接调用。当子类覆写了同名方法后，使用 `super.method()` 可以显式调用父类版本。

```
module junk();
class base;
virtual function void greet();
   $display("Hello from Base");
endfunction

function void base_only();
   $display("This method is only in Base");
endfunction
endclass

class extended extends base;
virtual function void greet();
   super.greet();           // 显式调用父类 greet
   $display("Hello from Extended");
endfunction

function void call_base_only();
   base_only(); // 直接调用继承的父类方法
endfunction
endclass

initial begin
extended ext_obj = new();
ext_obj.greet();
ext_obj.call_base_only();
```

```
end
endmodule

//Hello from Base
//Hello from Extended
//This method is only in Base
```

- 父类句柄调用子类方法
- 父类句柄**不能**直接调用子类中新定义（父类中不存在）的方法。若需要调用，必须先用 `$cast` 向下转型为子类类型。
- 但对于父类中声明为 `virtual` 的方法，通过父类句柄调用时会动态绑定到子类的覆写版本（见第2节）。

```
module junk();
class base;
virtual function void greet();
   $display("Hello from Base");
endfunction
endclass

class extended extends base;
virtual function void greet();
   $display("Hello from Extended");
endfunction

function void extended_only();
   $display("Extended-only method called");
endfunction
endclass

initial begin
extended ext_obj = new();
base base_handle;
extended cast_obj;

base_handle = ext_obj;

base_handle.greet(); // virtual, 动态绑定, 调用子类 greet
// base_handle.extended_only(); // 非法！父类句柄不能调用子类新方法

// $cast 向下转型后可调用子类新方法
if ($cast(cast_obj, base_handle)) begin
   cast_obj.extended_only(); // 合法
end
end
endmodule

//Hello from Extended
//Extended-only method called
```

- `local` 方法：仅限父类内部调用

- `local` 修饰的方法只能在定义该方法的类内部调用，子类和外部均无法访问。

```
module junk();
class base;
local function void secret();
   $display("Base local method called");
endfunction

function void public_call();
   secret(); // 父类内部调用 local 方法合法
endfunction
endclass

class extended extends base;
function void try_call();
   // secret(); // 非法！子类不能调用父类 local 方法
   public_call(); // 通过父类公共方法间接调用
endfunction
endclass

initial begin
extended ext_obj = new();
ext_obj.try_call();
end
endmodule

//Base local method called
```

- `protected` 方法：父类和子类内部可调用，外部不可调用

```
module junk();
class base;
protected function void helper();
   $display("Base protected helper called");
endfunction
endclass

class extended extends base;
function void do_work();
   helper(); // 子类内部调用父类 protected 方法，合法
endfunction
endclass

initial begin
extended ext_obj = new();
ext_obj.do_work();
// ext_obj.helper(); // 非法！外部不可调用 protected 方法
end
endmodule
```

```
//Base protected helper called
```

- Summary

访问场景	public 方法	protected 方法	local 方法
父类内部	✓	✓	✓
子类内部（直接继承/super）	✓	✓	✗
类外部通过句柄	✓	✗	✗
父类句柄调用子类新方法	✗（需 $cast）	✗	✗
父类句柄调用 virtual 覆写方法	✓（动态绑定）	✓（动态绑定）	✗